

Chiến Lược Triển Khai Hệ Thống Video Evidence Retrieval — AI Challenge 2026

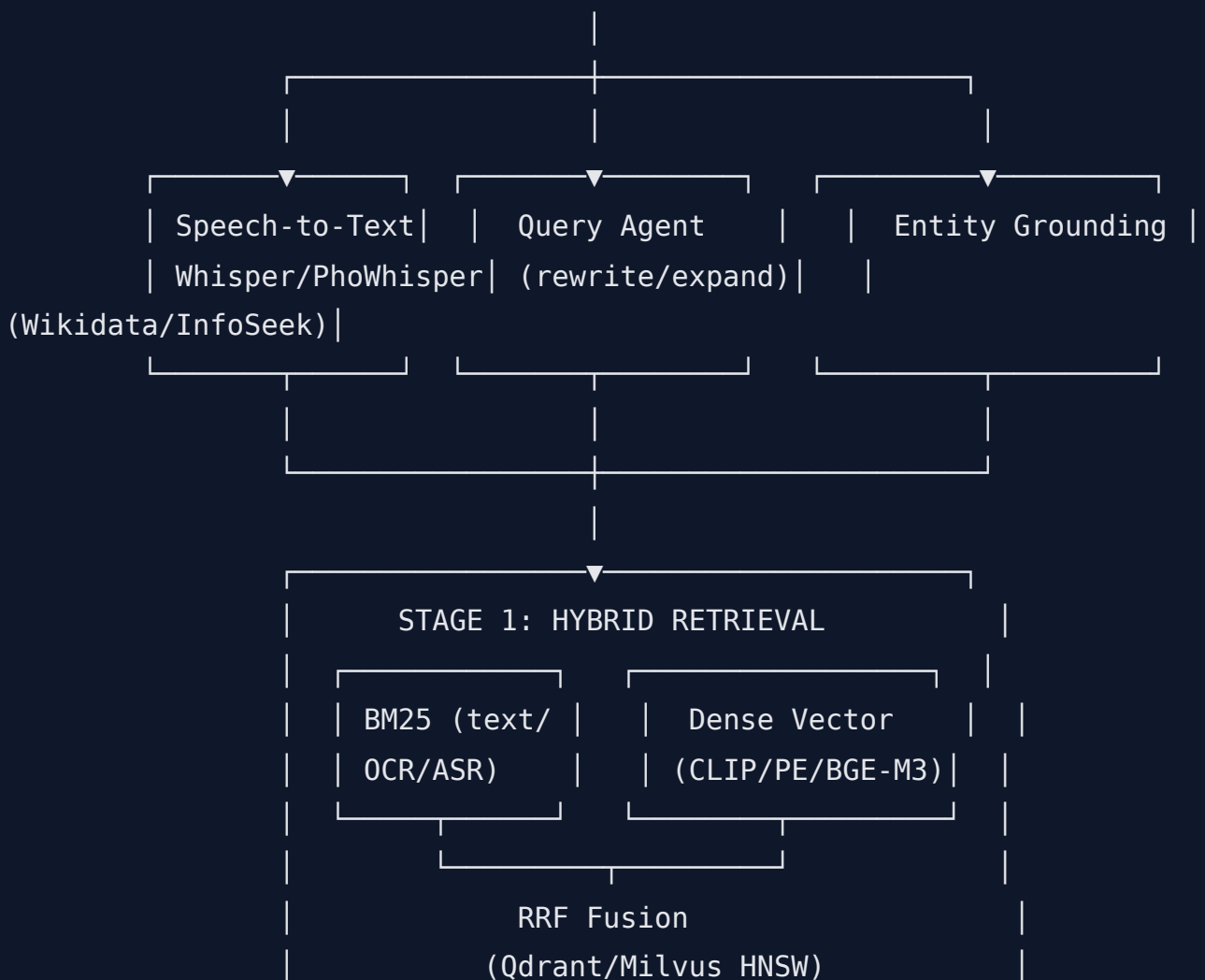
Sơ Đồ Kiến Trúc Hệ Thống Tổng Quan

FRONTEND (React + Vite)

Grid thumbnail | Video/Image viewer | Voice input | Keyboard shortcuts
Verifier badge | Temporal cluster scroll | DRES submit panel

REST / WS

API GATEWAY (FastAPI - async)



- Load `.npy` CLIP features (clip-ViT-B-32) của BTC vào Vector DB
- Chuẩn hóa Transcript (`audio_xxx.txt`) + OCR (`ocr.json`) → **Segments Schema** thống nhất theo timestamp
- Triển khai Hybrid Text Search (BM25 + Dense + RRF)
- Dựng Search UI Prototype (grid thumbnail, latency thấp)

So sánh các kỹ thuật

Kỹ thuật	Vai trò	Điểm mạnh	Điểm yếu
BM25 (sparse)	Tìm kiếm từ khóa OCR/Transcript	Chính xác cho tên riêng, số liệu, mã hiệu; không cần GPU; nhanh	Không hiểu ngữ nghĩa, fail khi câu hỏi diễn đạt khác từ gốc
bge-m3 (dense, multilingual)	Embedding text cho Vector Search	Hỗ trợ tiếng Việt tốt, kiêm cả dense+sparse+multi-vector (ColBERT-style) trong 1 model, mạnh trên benchmark đa ngôn ngữ	Model lớn (~2.2GB), latency embedding cao hơn model nhỏ
paraphrase-multilingual-mpnet	Embedding text thay thế	Nhẹ, nhanh, dễ deploy	Yếu hơn bge-m3 về retrieval đa ngôn ngữ, đặc biệt tiếng Việt chuyên ngành
RRF (Reciprocal Rank Fusion)	Gộp kết quả BM25 + Dense	Không cần train, không cần tinh chỉnh trọng số phức tạp, robust	Không tối ưu theo domain cụ thể, có thể cần weighted-RRF khi 1 nguồn yếu hơn (như Solution Notes ghi: Metadata/OCR search "rất tệ" nên cần trọng số nhỏ)
Qdrant	Vector DB	Dễ deploy (Docker), filter theo metadata mạnh, hỗ trợ HNSW + payload filtering tốt cho Verifier Agent sau này	Throughput thấp hơn Milvus ở scale rất lớn (nhưng dataset này không quá lớn nên không vấn đề)

Milvus	Vector DB thay thế	Scale tốt hơn ở cụm lớn, nhiều index types	Setup phức tạp hơn (cần etcd, MinIO), overkill cho quy mô thi đấu
HNSW index	ANN search	Latency < 100ms, recall cao	Tốn RAM hơn IVF, cần tune <code>ef_search / M</code>

✔ **Lựa chọn tốt nhất: bge-m3 + Qdrant (HNSW) + BM25 + RRF**

Giải thích: bge-m3 là lựa chọn tối ưu vì hỗ trợ tiếng Việt mượt (dataset tin tức tiếng Việt), và đặc biệt model này tích hợp sẵn cả **dense + sparse (lexical) + multi-vector** trong một forward pass — nghĩa là có thể dùng chính bge-m3 để sinh cả vector dense lẫn sparse weights, giảm độ phức tạp pipeline so với việc maintain 2 model riêng. Qdrant được chọn vì setup đơn giản bằng Docker (phù hợp với deadline gấp), payload filtering mạnh (cần thiết cho Verifier Agent ở Giai đoạn 3 khi cần filter theo `object_class`, `video_id`, `timestamp`). RRF là baseline gộp rank tốt nhất để bắt đầu, sau đó có thể chuyển sang weighted-RRF khi có dữ liệu eval thực tế (đúng như team năm ngoái nhận ra Metadata search cần trọng số nhỏ hơn).

GIAI ĐOẠN 2: Truy Xuất Đa Phương Thức & Tái Xếp Hạng Hai Giai Đoạn

Mục tiêu: Tích hợp Visual Search, nâng từ "tìm thô" lên "Two-stage Retrieval" để giải bài toán precision + latency.

Việc cần làm:

- Mở rộng Keyframes: tự cắt thêm frame từ video gốc + trộn keyframe BTC + de-duplication
- Visual Search Baseline: dùng CLIP `.npy` của BTC → lấy top 100-200 candidates
- Two-stage Reranking: Cross-Encoder (Object info từ Faster R-CNN JSON)
- Speech-to-Text cho input bằng giọng nói + LLM Query Expansion

So sánh các kỹ thuật

Kỹ thuật	Vai trò	Điểm mạnh	Điểm yếu
----------	---------	-----------	----------

CLIP ViT-B/32 (có sẵn từ BTC)	Stage-1 visual filter	Đã có sẵn .npy , không tốn compute lại, nhanh	Độ chính xác thấp với câu hỏi phức tạp/mơ hồ, embedding ngữ nghĩa nông
BEiT-3	Visual embedding nâng cao	Multi-modal pretraining mạnh, tốt cho region-level + global	Cần tính lại feature cho toàn bộ keyframe → tốn GPU/thời gian
InternVideo2	Video-level embedding	Hiểu được temporal/hành động liên tục → tốt cho TRAKE & Visual KIS	Model lớn, infer chậm, cần GPU mạnh để chạy realtime
Meta Perception Encoder (PE)	Visual embedding (theo kinh nghiệm năm ngoái)	Theo Solution Notes: model "rất mạnh", outperform metadata search	Cần tính lại toàn bộ feature, chi phí compute
BGE-reranker	Cross-encoder text reranking	Nhẹ, nhanh, độ chính xác cao cho text-text matching	Không tự "nhìn" được ảnh — chỉ hiểu qua caption/OCR/object list
Gemini Flash (LVLM Cross-Encoder)	Reranker đa phương thức	Hiểu cả ảnh + text, output CoT giải thích → hữu ích cho QA, chi phí thấp, tốc độ tốt	Phụ thuộc API (rate limit, chi phí khi scale), latency mạng
Qwen2-VL / LLaVA-1.6	LVLM Cross-Encoder (local)	Chạy local, không lo rate limit, có thể fine-tune	Cần GPU VRAM lớn, chậm hơn API nếu không tối ưu batch
Whisper-large	Speech-to-Text	Độ chính xác cao, đa ngôn ngữ	Model nặng, latency cao trên CPU
PhoWhisper	Speech-to-Text tiếng Việt	Tối ưu riêng cho tiếng Việt, chính xác hơn Whisper với accent Việt	Cộng đồng/support nhỏ hơn Whisper gốc

✔ **Lựa chọn tốt nhất: CLIP (stage-1 nhanh) + Two-stage rerank bằng BGE-reranker (text) + Gemini Flash làm LVLM Cross-Encoder, PhoWhisper cho voice**

Giải thích: Giữ CLIP .npy có sẵn của BTC làm bộ lọc stage-1 (top 100-200) để tiết kiệm compute — đúng với khuyến nghị "UI > Method > Model" trong Solution Notes, nghĩa là không cần đầu tư quá nhiều vào việc tính lại embedding nặng (BEiT-3/InternVideo2) ngay từ đầu nếu thời gian gấp. Ở stage-2, kết hợp BGE-reranker (rẻ, nhanh, dùng cho text-only signals như OCR/transcript match) với Gemini Flash làm LVLM Cross-Encoder — vì Gemini Flash có chi phí thấp, tốc độ phản hồi tốt, và quan trọng nhất là **xuất được Chain-of-Thought (CoT)** giúp Verifier Agent ở Giai đoạn 3 và người dùng (Track 2) duyệt nhanh hơn. PhoWhisper được chọn cho Speech-to-Text vì tối ưu riêng tiếng Việt — phù hợp ngữ cảnh thí sinh Việt Nam đọc to câu hỏi.

Nâng cấp tùy chọn (nếu còn thời gian/GPU dư): tính thêm Perception Encoder hoặc InternVideo2 cho các câu Visual KIS/TRAKE — nơi CLIP yếu nhất.

GIAI ĐOẠN 3: Phân Tích Ngữ Cảnh Nâng Cao & Kiến Trúc Agentic RAG

Mục tiêu: Biến hệ thống thành Multi-Agent có khả năng tự sửa lỗi (CRAG) và xác thực (Fact-Checking) — chống penalty -100 điểm khi nộp sai.

Việc cần làm:

- Entity Grounding: liên kết tên riêng/địa danh với Wikidata, InfoSeek
- Corrective RAG (CRAG): Reflection — confidence thấp → tự sửa query, search lại
- Verifier Agent: đối chiếu Câu hỏi ↔ OCR ↔ Objects JSON, hạ rank nếu object không khớp
- Temporal Clustering: gom frame cùng segment ($\epsilon \approx 10\text{-}15$ frames)

So sánh các kỹ thuật

Kỹ thuật	Vai trò	Điểm mạnh	Điểm yếu
----------	---------	-----------	----------

LangGraph	Điều phối Multi-Agent	Graph-based, dễ debug từng node, hỗ trợ loop (cần cho CRAG reflection), kiểm soát state rõ ràng	Cần thiết kế graph cẩn thận, học curve ban đầu
AutoGen	Điều phối Multi-Agent	Conversation-based, nhanh prototype agent-to-agent	Khó kiểm soát luồng tuyến tính có điều kiện phức tạp (loop reflection), debug khó hơn LangGraph
CRAG (Corrective RAG)	Self-correction	Giảm hallucination, tự retry khi confidence thấp → giảm penalty nộp sai	Tăng latency (re-query), cần threshold tuning cẩn thận để không loop vô hạn
Wikidata / InfoSeek grounding	External knowledge	Giải quyết câu hỏi có thực thể cụ thể (tên người, địa danh) mà model không biết	Cần API call ngoài → latency + cần fallback khi API down
Verifier Agent (Object cross-check)	Fact-checking	Dùng dữ liệu Objects JSON (Faster R-CNN) có sẵn từ BTC, không cần model thêm — chỉ logic so khớp	Object detector OpenImagesV4 có thể miss class hiếm/blur → false negative khi hạ rank ảnh đúng
Temporal Clustering (ϵ-based)	UI grouping	Thuật toán đơn giản (sliding window theo timestamp), giảm nhiễu trùng lặp	Cần tune ϵ theo fps thực tế của video (25-35fps)

✅ Lựa chọn tốt nhất: LangGraph cho điều phối + CRAG reflection loop + Verifier Agent dựa trên Objects JSON (rule-based, không cần model mới)

Giải thích: LangGraph được chọn vì CRAG cần một **loop có điều kiện** (nếu confidence thấp → quay lại bước query rewriting) — đây chính xác là use-case mà LangGraph (dựa trên state graph với cạnh có điều kiện) xử lý tốt hơn AutoGen (thiên về hội thoại tuần tự giữa agent). Verifier Agent nên được implement **rule-based trước** (so khớp class object trong JSON với từ khóa câu hỏi qua dictionary đồng nghĩa, có hỗ trợ từ Wikidata khi cần) thay vì train model mới — vừa nhanh, vừa minh bạch, dễ debug khi sai, và tận dụng triệt để dữ liệu Objects đã có sẵn từ BTC. Đây là lớp bảo vệ rẻ nhất nhưng hiệu quả cao nhất để giảm penalty -100.

GIAI ĐOẠN 4: Tối Ưu Hệ Thống & Thi Đấu Thực Chiến (Production)

Mục tiêu: Đóng gói Docker, tối ưu latency end-to-end < 500ms, hoàn thiện 2 track thi đấu (Fully Auto M2M & Semi-Auto).

Việc cần làm:

- Async I/O cho đọc `.npy` CLIP features
- Tune ANN index (FAISS/HNSW): cân bằng RAM vs tốc độ
- Track 1 (M2M): tự nhận đề → Agentic RAG → top-1 confidence cao nhất → auto-submit DRES, đo Precision@K, mAP
- Track 2 (Semi-Auto): Keyboard shortcuts, filter panel, mục tiêu ≤2 click để submit
- Error Analysis: test trên VBS-Archive, Docker local giả lập DRES, tune lại trọng số các luồng search

So sánh các kỹ thuật

Kỹ thuật	Vai trò	Điểm mạnh	Điểm yếu
FAISS	ANN index	Cực nhanh, nhiều loại index (IVF, HNSW, PQ), tối ưu RAM tốt với PQ	Không có filter theo metadata mạnh như Qdrant — khó kết hợp Verifier filter
HNSW (trong Qdrant)	ANN index	Đã dùng từ Giai đoạn 1, filter + search cùng lúc	RAM cao hơn FAISS-PQ ở dataset rất lớn
Docker + Docker Compose	Đóng gói	Deploy đồng nhất mọi máy phòng thi, dễ rollback	Cần test kỹ trên môi trường offline (không internet) trước ngày thi
motmetrics / custom Python	Đo lường	Tính Recall@100, Precision@K theo chuẩn, dễ tích hợp CI	Cần viết script custom cho mAP/TRAKE accuracy theo công thức riêng của BTC

✔ **Lựa chọn tốt nhất: Giữ Qdrant/HNSW (đã setup từ GĐ1) + Docker Compose + script đo lường custom theo công thức**
Score = max(0, 500 - 100*penalty + f(time))

Giải thích: Không nên đổi sang FAISS ở giai đoạn cuối vì sẽ phải viết lại toàn bộ filter logic (Verifier Agent cần filter theo `object_class` , `video_id` — thứ Qdrant đã làm tốt từ đầu). Việc đổi index ở giai đoạn cuối tạo rủi ro bug cao, trái với nguyên tắc "ưu tiên không miss câu nào" trong Solution Notes. Thay vào đó, dồn lực vào: (1) Docker Compose để đảm bảo chạy được offline 100% trong phòng thi (rất quan trọng vì M2M Track cần kết nối ổn định với DRES server của BTC), và (2) script đo lường **đúng theo công thức điểm của BTC** (`Score = max(0, 500 - 100×penalty + f(time))`) để Error Analysis phản ánh chính xác điểm số thật, từ đó tune trọng số RRF/Reranker dựa trên test-cases VBS-Archive.

Bảng Tổng Hợp Stack Đề Xuất

Layer	Công nghệ chọn
Frontend	React + Vite
Backend	FastAPI (async)
Vector DB	Qdrant (HNSW)
Text Embedding	bge-m3
Sparse Search	BM25
Fusion	RRF → weighted-RRF (sau khi có eval data)
Visual Embedding (stage-1)	CLIP ViT-B/32 (BTC cấp sẵn)
Reranker text	BGE-reranker
Reranker đa phương thức	Gemini Flash (LVLM)
Speech-to-Text	PhoWhisper
Multi-Agent orchestration	LangGraph
Self-correction	CRAG (Reflection loop)
Fact-checking	Verifier Agent (rule-based + Objects JSON + Wikidata grounding)

Đóng gói	Docker + Docker Compose
Đo lường	Custom script theo <code>f(time)</code> , Score , Recall@100, mAP

Tài Liệu Tham Khảo Để Nghiên Cứu Thêm

Mô hình Embedding & Retrieval

- **BGE-M3**: BGE M3-Embedding (Hybrid, Multilingual, Multi-Granularity) — <https://arxiv.org/abs/2402.03216>
- **CLIP**: Learning Transferable Visual Models From Natural Language Supervision — <https://arxiv.org/abs/2103.00020>
- **BEiT-3**: Image as a Foreign Language (BEiT-3) — <https://arxiv.org/abs/2208.10442>
- **InternVideo2**: Scaling Foundation Models for Multimodal Video Understanding — <https://arxiv.org/abs/2403.15377>

Reranking

- **BGE-Reranker / FlagEmbedding repo** — <https://github.com/FlagOpen/FlagEmbedding>
- **Reciprocal Rank Fusion (RRF)** gốc: Cormack et al., "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods" — <https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf>

Agentic RAG / Self-Correction

- **CRAG (Corrective Retrieval Augmented Generation)** — <https://arxiv.org/abs/2401.15884>
- **Self-RAG**: Learning to Retrieve, Generate, and Critique through Self-Reflection — <https://arxiv.org/abs/2310.11511>
- **LangGraph docs** — <https://langchain-ai.github.io/langgraph/>
- **AutoGen docs** — <https://microsoft.github.io/autogen/>

Multimodal Search Agents

- **MMSearch / MMSearch-Plus**: Search engine for multimodal AI — tìm "MMSearch benchmark arxiv" để cập nhật bản mới nhất
- **Set-of-Mark (SoM) Prompting** — <https://arxiv.org/abs/2310.11441>

Entity Grounding

- **InfoSeek:** A Visual Question Answering Dataset for Visual Entity Recognition — <https://arxiv.org/abs/2302.11713>
- **Wikidata Query Service** — <https://query.wikidata.org/>

Speech-to-Text

- **Whisper (OpenAI)** — <https://arxiv.org/abs/2212.04356>
- **PhoWhisper** — <https://github.com/VinAIResearch/PhoWhisper>

Vector DB & ANN

- **HNSW:** Efficient and robust approximate nearest neighbor search — <https://arxiv.org/abs/1603.09320>
- **Qdrant docs** — <https://qdrant.tech/documentation/>
- **FAISS** — <https://github.com/facebookresearch/faiss>

Đánh giá / Object Detection

- **Faster R-CNN** — <https://arxiv.org/abs/1506.01497>
- **OpenImagesV4** — <https://storage.googleapis.com/openimages/web/index.html>